BIET TRANSPORT APPLICATION

Project Report Submitted to

Jawaharlal Nehru Technological University Hyderabad

In partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING

By

GATADI ROSHAN	21E11A05B2
RAMAVATH VAMSHI	21E11A05F0
GOURA MAHINDRA	21E11A05B3

Under the guidance of

Dr Rama Prakasha Reddy.Ch
Assistant Professor
Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY
(An Autonomous Institution)
Accredited by NAAC, 'A' Grade, Accredited by NBA (UG Programmes: CSE, ECE)
Approved by AICTE, Affiliated to JNTUH Hyderabad
Ibrahimpatnam -501 510, Hyderabad, Telangana.

JUNE 2025

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY

(An Autonomous Institution)

Accredited by NAAC, 'A' Grade, Accredited by NBA (UG Programmes: CSE, ECE)

Approved by AICTE, Affiliated to JNTUH Hyderabad,

Ibrahimpatnam -501 510, Hyderabad, Telangana.

Certificate

*This is to certify that the Project work (phase-2) entitled "***BIET TRANSPORT APPLICATION***" is the Bonafide work done*

By

GATADI ROSHAN	**21E11A05B2**
RAMAVATH VAMSHI	**21E11A05F0**
GOURA MAHINDRA	**21E11A05B3**

in the Department of Computer Science and Engineering, ***BHARAT INSTITUTE OF ENGINEERING AND TECHNOLOGY****, Ibrahimpatnam is submitted to* ***Jawaharlal Nehru Technological University, Hyderabad*** *in partial fulfilment of the requirements for the award of* ***B.Tech*** *degree in* ***Computer Science and Engineering*** *during* ***2024- 2025.***

Supervisor:
Dr.Rama Prakasha Reddy.Ch
Assistant Professor
Dept of Computer Science and Engineering
Bharat Institute of Engineering and Technology
Ibrahimpatnam–501 510, Hyderabad

Head Of The Department
Dr. Deepak Kachave
Professor
Dept of Computer Science and Engineering
Bharat Institute of Engineering and Technology
Ibrahimpatnam–501 510, Hyderabad

Viva-Voce held on.....

Internal Examiner **External Examiner**

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of the task would be put incomplete without the mention of the people who made it possible, whose constant guidance and encouragement crown all the efforts with success.

We avail this opportunity to express our deep sense of gratitude and hearty thanks to **Sri CH. Venugopal Reddy**, Chairman of BIET, for providing congenial atmosphere and encouragement.

We would like to thank **Prof. G. Kumaraswamy Rao**, Former Director & O.S. of DLRL Ministry of Defence, Sr. Director R&D, BIET, and **Dr. V Srinivasa Rao**, Professor Dean CSE, for having provided all the facilities and support.

We would like to thank our Department In charge **Dr. Deepak Kachave**, for encouragement at various levels of our Project.

We are thankful to our Project Coordinator **Dr. Rama Prakasha Reddy.Ch**, Assistant Professor, Computer Science and Engineering for his support and cooperation throughout the process of this project.

We are thankful to our guide **Dr. Rama Prakasha Reddy.Ch**, Assistant Professor, Computer Science and Engineering for his sustained inspiring Guidance and cooperation throughout the process of this project. His wise counsel and suggestions were invaluable.

We express our deep sense of gratitude and thanks to all the Teaching and Non-Teaching Staff of our college who stood with us during the project and helped us to make it a successful venture.

We place highest regards to our Parent, our Friends and Well-wishers who helped a lot in making the report of this project

GATADI ROSHAN 21E11A05B2
RAMAVATH VAMSHI 21E11A05F0
GOURA MAHINDRA 21E11A05B3

Declaration

We hereby declare that this Project (phase-2) is titled **BIET TRANSPORT APPLICATION** is a genuine Project work carried out by us, in **B.Tech (Computer Science and Engineering)** degree course of **Jawaharlal Nehru Technology University Hyderabad, Hyderabad** and has not been submitted to any other course or university for the award of my degree by me.

Signatures of the Project team members

1.

2.

3.

ABSTRACT

This Project titled “BIET Transport Application” is a smart, role-based transportation management system developed to streamline and digitize campus bus operations for students, faculty, and administrators. This application provides real-time GPS tracking, estimated arrival times, live route updates, and intelligent notifications, aiming to enhance the safety, efficiency, and convenience of institutional commuting.

The system is built on a modular architecture, using Firebase for real-time data handling, while ensuring secure communication between mobile clients and the backend via RESTful APIs. A role-based access control mechanism allows users to access features tailored to their roles—students can view live bus locations and receive ETA alerts, faculty can monitor specific routes, and admins can manage buses, users, and route data through a dedicated dashboard.

An integrated QR code system enables secure user verification before boarding, linked with payment status to ensure only authorized users access the service. The sequence diagram outlines key operations such as login, QR validation, location update, and admin actions, reflecting the streamlined data flow within the application.

By promoting transparency, reducing wait times, and enhancing operational control, the Biet Transport App contributes to a more intelligent and sustainable campus transport system aligned with modern digital infrastructure goals.

Keywords: Mobile Transport Solution, Role Based access, Estimated time of arrival, Gps-tracking system etc.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Acknowledgements		iii
Abstract		v
Table of Contents		vi
List of Figures		vii
List of Tables		viii
List of Symbols and Abbreviations		ix
1.	INTRODUCTION	1
1.1	Introduction To BIET Transport App	1
1.2	App Components	2
1.3	Activating components	4
2.	RELATED WORK	6
3.	MOTIVATION	7
4.	OBJECTIVE	8
5.	PROBLEM STATEMENT	10
6.	DESIGN METHODOLOGY	11
6.1	Architecture	11
6.2	Modules	12
6.3	Requirements and Specifications	14
6.4	UML Diagrams	15
7.	EXPERIMENTAL STUDIES	20
7.1	Overview	20
7.2	Stages Of Testing	20
7.3	Test Cases	21
7.4	Results Analysis	24
8.	CONCLUSION AND FUTURE SCOPE	46
	References	47

LIST OF FIGURES

Figure No.	Caption	Page No.
Figure - 6.1	System Architecture of BIET Transport Application	11
Figure - 6.2	Use case diagram BIET Transport Application	15
Figure - 6.3	Class diagram BIET Transport Application	16
Figure - 6.4	Activity diagram BIET Transport Application	17
Figure - 6.5	Sequence diagram BIET Transport Application	18
Figure - 6.6	State-Chart diagram BIET Transport Application	19
Figure - 7.1	Login page	42
Figure - 7.2	Sign-Up page	42
Figure - 7.3	Registration page of different roles	43
Figure - 7.4	Home page	43
Figure - 7.5	Buses-info page	44
Figure - 7.6	Qr , Scanner page	44
Figure - 7.7	Profile pages of different roles	45
Figure - 7.8	Admin management page	45

LIST OF TABLES

Table No.	Caption	Page No.
Table - 7.1	Authentication & Role Management	21
Table - 7.2	Admin Functionality	22
Table - 7.3	Bus Information & live Tracking	22
Table - 7.4	Qr code Generation & Verification	22
Table - 7.5	Profile Management	23
Table - 7.6	Registration and Error 1	23
Table - 7.7	Test Case for Login	24
Table - 7.8	Test Case for Registration	25
Table - 7.9	Test Case for Adding Bus	26
Table - 7.10	Test Case for qr code (student panel)	27
Table - 7.11	Test Case for qr code verification (admin / Faculty panel)	28
Table - 7.12	Test Case for Adding of users into list	29

LIST OF SYMBOLS AND ABBREVIATIONS

LIST OF ABBREVIATIONS

Symbol	Description
BIET	Bharat Institute of Engineering and Technology
APK	Android Application Package
API	Application Programming Interface
GPS	Global Positioning System
QR code	Quick Response code
RBAC	Role- Based Access Control
CRUD	Create, Read, Update, Delete

1. INTRODUCTION

1.1 Introduction To BIET Transport app

The Biet Transport App is a comprehensive and dedicated transport management system developed for the Bharat Institute of Engineering and Technology (BIET) to improve the daily commute experience for students and faculty. With many individuals traveling from various parts of the region, the need for a centralized and accessible transport system became evident. Traditional transport methods were prone to confusion due to unclear information about routes, stops, and fee structures. The app was developed to address these challenges by offering a user-friendly digital solution that brings greater clarity, convenience, and organization to the college's transportation services.

A significant problem faced by commuters was the lack of transparency in bus route numbers, stop details, and the process of paying transportation fees. Many users were unsure of how to pay track pending payments, leading to missed deadlines and inefficient communication. The Biet Transport App resolves these issues by displaying clear information about all routes and stops, along with payment verification option of transportation fees. This system replaces outdated manual processes and ensures that students and staff can manage their transport details directly from their mobile devices.

One of the key features of the app is real-time GPS tracking, allowing users to view the current location of their assigned buses. This eliminates the uncertainty of waiting for buses and helps commuters plan their journey more accurately. The app also provides a detailed view of bus routes using maps, making it easier for users to identify the nearest stop and monitor the vehicle’s movement. These capabilities make the transportation system more efficient and user-friendly, especially for new users who are unfamiliar with the routes.

The Biet Transport App also offers a clean and intuitive interface tailored to different types of users. From the login and sign-up pages, users can register according to their role—student, faculty, or admin. Once logged in, users can access features like bus route details, maps, QR scanning, and personal profile information. Administrators have access to additional controls such as managing bus data and overseeing operations. By digitizing these processes, the app reduces dependency on manual record-keeping, improves transport fee management, and enhances overall campus mobility. This solution represents a step toward smarter, more efficient transport infrastructure within the academic environment.

1.2 App Components

App components are the fundamental building blocks of the Biet Transport App. Each component serves a unique function in the application and helps define its behavior. Although not all components directly interact with the user, they play vital roles in delivering a complete and seamless transportation management experience. Each one is implemented using Android component classes and supports the overall structure and flow of the app. The Biet Transport App utilizes a combination of Android's main app components to provide real-time tracking, digital payments, and user-specific features.

There are four types of app components commonly used in Android development. Below is a description of how each is implemented in the Biet Transport App.

Activities

The **BIET Transport App** provides a structured and user-friendly experience through its various activities, each designed to perform a specific role in the transport system. The **Login Activity** serves as the secure entry point, verifying users through their email and password, and directing them to appropriate sections of the app based on their roles — whether they are students, faculty, or admins. This ensures that access to app features is tightly controlled and personalized. Complementing this is the **Sign-Up Activity**, which allows new users to register under a selected role by collecting essential details like name, email, and password. Once registered, their data is saved to the database and they are granted access according to their profile. The **Bus Route Activity** provides comprehensive details of available bus routes including stop names, timings, and the assigned buses. It enables students and faculty to plan their commute effectively while allowing admins to update or monitor route data for better scheduling and service management. The **Map Activity** offers a real-time, visual representation of bus locations using GPS data, showing nearby stops and estimated arrival times to help users make informed decisions and reduce unnecessary waiting.[10]

In addition to these interactive activities, the app is powered by several essential background **services** that manage data flow, security, and system updates. The **GPS Tracking Service** continuously collects the real-time coordinates of buses using the device's GPS and updates the local database, which the map interface then uses to show live tracking. The **Authentication Service** ensures only legitimate users can log in and access the app, assigning roles and managing session persistence securely. To support offline functionality and improve performance, the **Local Data Sync Service** manages all vital data such as user profiles, bus

details, route information, and payment status within the local database. The **Map Rendering Service** then takes this data and integrates it with APIs like Google Maps to render visually accurate, interactive maps. The **Admin Data Update Service** empowers administrators to make critical updates such as modifying routes, changing bus stop timings, or updating student payment statuses. One key feature tied to this is the **QR Verification & Entry Service**, where students marked as "paid" receive a unique QR code. This code must be scanned when boarding the bus, and only verified students are allowed entry. This feature ensures secure and fair use of the transport service, preventing unpaid users from accessing it.[3]

Together, these well-coordinated activities and background services provide a robust and seamless experience for users. Whether it's navigating through different screens, tracking buses in real time, or ensuring secure and role-based access, the BIET Transport App delivers a reliable and efficient solution for campus transportation management

Content Providers

In the Biet Transport App, Content Providers serve as a centralized interface to access and manage structured data stored in the local/cloud database, such as user profiles, bus route information, stop locations, and payment statuses. They enable different components of the app—including the QR code scanner, map interface, and admin dashboard—to interact with shared data consistently and securely. For example, when verifying if a student is eligible to board a bus, the QR scanning module can access the student's payment status through the content provider. Although data is not shared with external apps, using content providers promotes clean architecture, modular development, and improved data integrity. It also simplifies data access logic by allowing components to query data using URIs, making the app easier to maintain and extend in the future.

Broadcast Receivers

In the Biet Transport App, **Broadcast Receivers** are used to handle system-wide events and trigger actions without direct user interaction. A key example is monitoring **network connectivity changes**—when the internet connection is lost, the broadcast receiver can notify users and pause real-time GPS updates. It may also resume services once connectivity is restored. Additionally, it can respond to events like device reboot or battery status, ensuring critical background tasks (e.g., location updates) restart when needed. These receivers are lightweight, run in the background, and help maintain app stability, especially in dynamic mobile environments with fluctuating network conditions.

1.3 Activating Components

The **BIET Transport App** is engineered with a powerful architecture comprising several activating components that enable it to operate dynamically, efficiently, and reliably across various user scenarios. These components are fundamental to both the user experience and the behind-the-scenes processes that drive the app's functionality. At the core of this system are **Activities**, which form the main structure of the user interface. These are the interactive screens users engage with directly, and each is designed to handle a specific task. For example, the login and registration activities allow users to securely authenticate themselves based on their role—student, faculty, or administrator. Once logged in, users can view detailed bus routes, scan QR codes for trip validation, and track buses in real time on a live map.

Each activity is purpose-built to ensure that users can perform their tasks intuitively and without confusion. This modular approach helps keep the user interface organized, responsive, and easy to navigate. Whether someone needs to scan a QR code to board the bus, check their travel history, or find the estimated time of arrival for a bus, these interactions are all handled seamlessly through well-designed activities.

While Activities manage what users see and do, the app also relies heavily on **Services**, which operate in the background. Services are essential for handling tasks that must continue running even when the user is not actively engaging with the app. For instance, the GPS tracking service continuously monitors the location of each bus and sends updates to the central server. This ensures that the map displayed to users always reflects real-time bus positions, without requiring them to refresh or reload any part of the app manually. Similarly, data synchronization services work silently to keep user data, trip logs, and schedules up-to-date by periodically syncing with the backend server.

Another vital component is the **Broadcast Receiver**, which listens for important system-level events. In mobile environments, connectivity can drop, devices can restart, and power-saving modes can be triggered. Broadcast Receivers enable the app to respond to these changes in real time. For example, if the device loses internet connection, the app can pause tracking and notify the user. When the connection is restored or the device reboots, the app can automatically resume critical background tasks without requiring user input. This ensures that essential functions like GPS tracking and data syncing are not interrupted unnecessarily.

In the background, **Content Providers** offer structured and secure access to the app's local database. They act as intermediaries that allow various parts of the app to share data consistently and safely. Whether it's verifying a student's payment status before allowing them to board the bus, or retrieving a list of available routes, Content Providers ensure that data flows efficiently and with proper access controls in place.

Together, these activating components—Activities, Services, Broadcast Receivers, and Content Providers—create a seamless and robust ecosystem. They ensure that the **Bharat Transport App** stays connected, responsive, and reliable, delivering essential features like secure login, real-time GPS tracking, QR-based student verification, and uninterrupted performance in varying mobile conditions. This architecture forms the technical backbone of an intelligent and user-friendly transport management solution.

2. RELATED WORK

In recent years, numerous transport management applications have emerged, aiming to streamline the commuting experience and offer real-time tracking capabilities. These applications are widely adopted across various sectors, including public transportation, corporate fleets, and educational institutions. Prominent examples such as Moovit, Citymapper, and Transit have become popular for offering features like live bus and train tracking, route planning, traffic predictions, and estimated arrival times. [1,2]

However, while these apps are excellent for general-purpose commuting, they are primarily designed for the broader public and often lack features tailored to the unique needs of specific institutions. For instance, they typically do not provide role-based access control or support user-specific logic, such as student status verification or transport fee tracking. In public apps, all users usually access the same interface and data, making them unsuitable for academic environments where students, faculty, and administrators require different levels of access and functionality. [3]

Within educational institutions, some colleges and universities have experimented with GPS-based transport tracking systems. These are often basic implementations that offer location tracking via Google Maps integrations. However, such systems commonly lack centralized administrative control, dedicated portals for different user roles, or real-time synchronization of critical data like student payment records. Most of these apps do not support secure boarding mechanisms, such as QR code verification linked to transport fee payment status, nor do they handle role-based login or localized offline data access.[4]

Furthermore, while cloud platforms like Firebase are frequently used for real-time data syncing and user authentication, they tend to be dependent on constant internet connectivity, which may not be ideal in all operational contexts.[8]

The BIET Transport Application addresses these limitations by providing a tailored transport management solution for academic institutions. It integrates real-time GPS tracking, role-based login (student, faculty, admin), QR code-based access control, and fee status verification, all managed through a centralized admin panel. This targeted approach offers a more secure, efficient, and institution-specific transport experience, setting it apart from generic public tracking systems.[7,10]

3. MOTIVATION

Efficient, safe, and well-organized transportation is a crucial necessity for educational institutions, particularly for colleges such as Bharat Institute of Engineering and Technology (BIET), where hundreds of students and faculty members rely on college-operated buses for their daily commute. Despite the importance of institutional transport systems, many colleges continue to depend on traditional, manually managed processes that often lack essential features like transparency, real-time tracking, and centralized control

Key issues typically found in conventional transport setups include uncertainty in bus arrival times, difficulty in accessing route information, manual fee status checking, and a lack of structured control over boarding authorization. These inefficiencies not only reduce student punctuality but also impact overall productivity, increase administrative burden, and pose potential risks to safety and accountability.

The Biet Transport App was conceptualized and developed to address these challenges through a comprehensive, smart, and digital solution. The primary motivation behind this project is to transform outdated transport practices into a streamlined, reliable, and technologically advanced system tailored for academic institutions. By integrating real-time GPS tracking, QR code-based entry validation, and role-based access control, the app significantly enhances both the commuter experience and administrative efficiency.

For students, the app offers features such as live bus tracking, estimated arrival times, and QR codes that are only generated after successful transport fee payment, ensuring that only verified students can board the buses. This reduces disputes at boarding points and removes the need for manual checks by staff.

For administrators, the app functions as a centralized control panel, enabling real-time monitoring of bus routes and positions, managing user access, verifying payment statuses, and updating schedules.

Unlike general-purpose tracking or navigation apps, the Biet Transport App is purpose-built for academic use and aligns closely with the logistical and operational needs of an educational transport environment. Ultimately, the motivation for this project is driven by the goal to improve efficiency, safety, and communication, making daily commuting smoother for students and significantly more manageable for institutional authorities.

4. OBJECTIVE

The Biet Transport App is a unified mobile-based transportation management solution designed to streamline and digitize the daily transport operations of students, faculty, and administrators within the Bharat Institute of Engineering and Technology (BIET). The primary objective is to provide a centralized platform that handles all aspects of college transport, improving transparency, efficiency, and user convenience.

The key goals of the app include:

The BIET Transport App is designed to streamline and digitize college transport operations by integrating students, faculty, and admins onto a single, unified platform. This integration simplifies route access, schedule management, and commuter monitoring, ensuring that all users interact efficiently within the same system. A key feature of the app is its **secure, role-based access**, which ensures data privacy and appropriate feature allocation—students can track buses and use QR codes for boarding, while admins manage critical functions like fee verification, route planning, and bus information updates.

To further enhance security and reduce manual overhead, the app employs **automated fee-based QR code access**, where only students who have completed their transportation fee payment receive a valid QR code for bus boarding. This approach eliminates the chances of unauthorized or fraudulent entries. The system also replaces manual data handling with **digitized, database-driven processes**, reducing errors in student records, fee validation, and route management. Additionally, the app includes **real-time GPS tracking**, allowing users to view the current location of their assigned bus through an interactive map interface. This helps students and faculty plan their departures more accurately, avoid long wait times, and improve overall punctuality and convenience.

FEASIBILITY STUDY

The feasibility study assesses the practicality of developing and deploying the Biet Transport App by analyzing key factors such as technical capabilities, resource availability, cost-effectiveness, and time constraints. It determines whether the app can be implemented successfully using current technologies like GPS, QR systems, and local databases. The study ensures the proposed solution is realistic, achievable, and beneficial for students, faculty, and administrators involved in Biet's transport management system.

1. TECHNICAL FEASIBILITY.
2. OPERATIONAL FEASIBILITY.
3. ECONOMICAL FEASIBILITY.
4. SCHEDULE FEASIBILITY.

TECHNICAL FEASIBILITY:

The technical requirements of the Biet Transport App are well within reach. The app will be built using proven technologies like Flutter or Android Studio for mobile development, along with Google Maps API for real-time GPS tracking. The backend can be managed using Firebase or a local database system like SQLite for storing user data, route information, and payment status. QR code generation and scanning are easily implementable using open-source libraries. All these tools and technologies are readily available and supported by modern development environments, making the system technically feasible.

OPERATIONAL FEASIBILITY

The app is designed to improve day-to-day transport management for students, faculty, and administrators. With simple interfaces, role-based access, and real-time data, users can quickly adapt to the system with minimal training. Admins can monitor transport activity via a web dashboard, students can scan QR codes to board, and faculty can track buses for timely commuting. The shift from manual to digital will reduce confusion, save time, and increase overall satisfaction—making the app highly feasible operationally.

ECONOMIC FEASIBILITY

The cost of development is moderate, requiring a small team of developers, cloud service subscriptions (if needed), and basic infrastructure for deployment. In return, the system reduces long-term operational costs by automating fee verification, route planning, and monitoring. The savings in administrative effort, paper usage, and travel management justify the investment, ensuring economic feasibility.

SCHEDULE FEASIBILITY

With proper planning, the app can be developed within a realistic timeframe. A phased development cycle—starting with core features like authentication, GPS tracking, and QR code scanning—ensures timely delivery.

5. PROBLEM STATEMENT

Despite the wide availability of generic bus tracking applications in the market, Bharat Institute of Engineering and Technology currently lacks a dedicated, institution-specific transport management system that fully addresses the unique operational needs of an academic environment. Existing applications offer only basic features such as GPS-based location tracking, which is insufficient for managing the complexities of a structured college transport network. Critical functions like role-based user access, real-time transport fee verification, and secure bus boarding mechanisms are notably absent, resulting in various operational challenges.

Students often encounter multiple issues under the current system. These include uncertainty about bus arrival times, limited or no communication regarding delays or route changes, and no real-time verification of their transport eligibility. This not only impacts the students' daily schedules but also raises security and accountability concerns.

Faculty and staff, who also depend on college transportation services, suffer similar inconveniences due to the lack of digital scheduling tools and real-time information. From an administrative standpoint, the absence of a centralized control system significantly increases the burden of manually tracking student payments, bus assignments, and user authentication. The lack of automation leads to higher chances of errors, reduced efficiency, and overall dissatisfaction among users.

To effectively address these challenges, the proposed Biet Transport App is designed as a comprehensive, integrated solution. It incorporates features such as real-time GPS tracking, QR code-based student verification, role-based access control, and a centralized admin dashboard. Only students who have successfully paid their transport fees will receive valid QR codes, ensuring that bus access is secure and restricted to eligible users. administrators gain powerful tools to monitor routes, verify users, manage fee records, and update schedules in real time.

By digitizing and centralizing the transport system, the app promises to enhance security, reduce manual workload, and offer a seamless commuting experience for students, faculty, and administrators alike. Ultimately, it aims to transform Biet's transport infrastructure into a smart, transparent, and highly efficient system tailored to the needs of an academic institution.

6. DESIGN METHODOLOGY

6.1 SYSTEM ARCHITECTURE

The System architecture consists of all the modules of the main module are shown below. However, it also shows how the modules access the resources and how they interact with each other and communicate with each other. It also shows the input, output and processing pathways of the application and also the flow of direction of input, output and processing directions of the application.

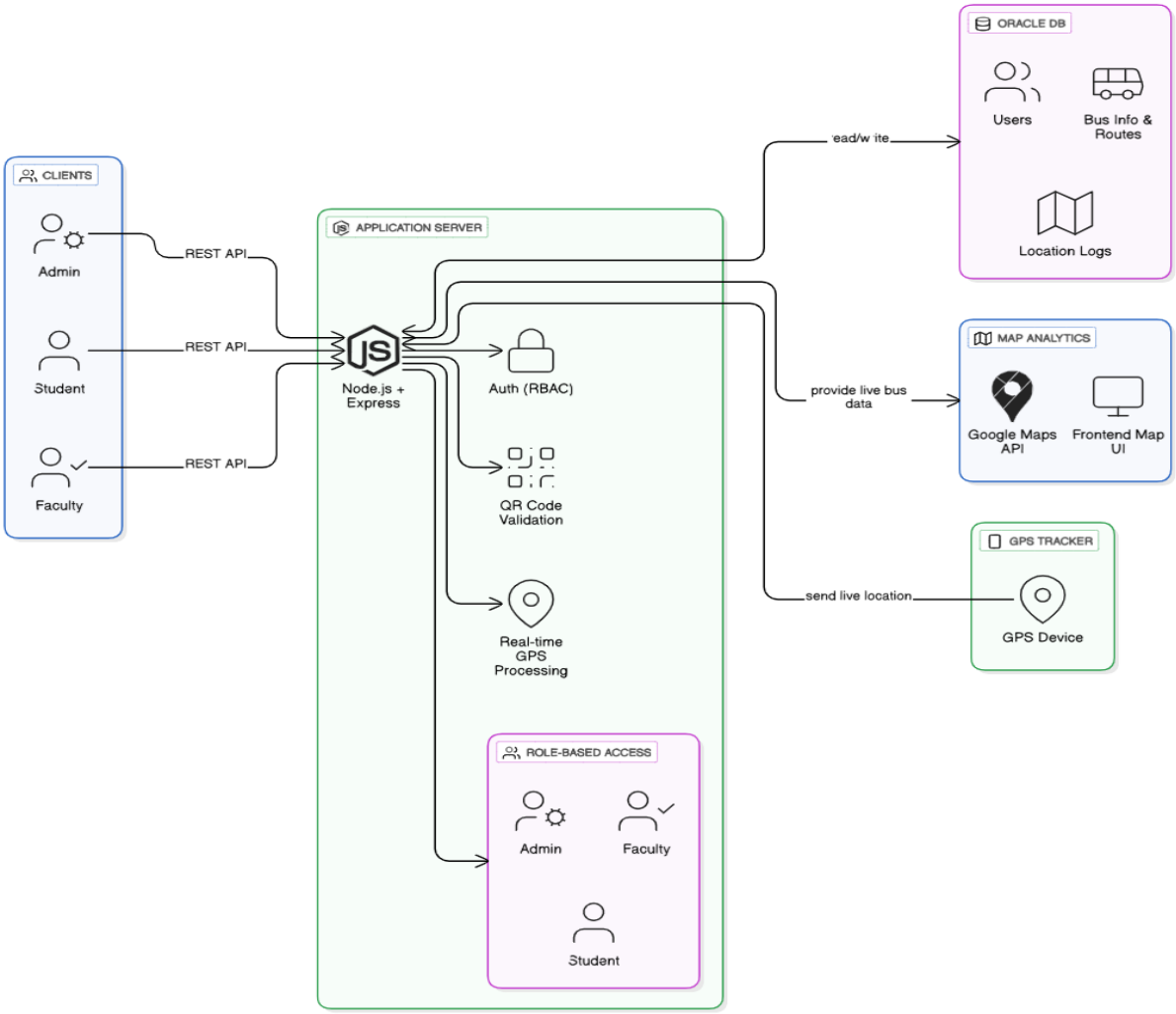


Figure 6.1: System architecture of BIET Transport Application

6.2 MODULES:

Login Page:

Login page is the initial screen that appears when the user launches the BIET Transport App. It is designed to ensure secure access by allowing users to select their role — such as *Student*, *Faculty*, or *Admin* — and enter their login credentials (ID and password). The login mechanism ensures that users are authenticated and redirected to features and interfaces appropriate to their role. It also includes error handling for incorrect credentials and provides a smooth gateway into the application's functionalities.

Registration Page:

If a user doesn't already have an account, they can access the **Registration Page** by clicking the "Sign In" or "Register" option on the Login Page. On this page, users can choose their respective role (Student, Faculty, Admin) and fill in the required personal details such as name, email, phone number, and password. After submitting the form, their data is securely saved to the app's database, and they gain access based on the permissions and features assigned to their role..

Home Page:

The **Home Page** provides a general overview and displays information related to the college. It can include important announcements, campus guidelines, transport updates, or quick access cards that navigate users to other key features in the app. It acts as a central dashboard where users land after login and serves as a hub for other interactions..

Buses Page:

The **Buses Page** displays comprehensive details about the available buses. It includes a list of bus numbers or names, the routes they follow, all scheduled stops, and estimated arrival and departure times for each stop. This page helps students and faculty plan their travel accordingly, reducing waiting times and confusion. The page may also allow real-time updates from admins in case of delays or route changes

Live Location Page:

The **Live Location Page** provides real-time GPS tracking of the buses using map integration (e.g., Google Maps API). This page allows students, faculty, and even admins to view the current

position of a specific bus, track its movement, and get accurate arrival predictions for their stop. This enhances the overall transport experience by reducing uncertainty and improving time management for commuters.

QR Page:

The **QR Page** plays a vital role in student verification. For students who have paid their bus fee, the app generates a unique QR code that appears on this page. This code must be shown at the time of boarding the bus. Faculty and Admins can use the built-in **QR Scanner** (also available on this page) to scan student QR codes to verify their validity. If a student is not marked as "paid" by the admin, the QR code will not be generated or will show as invalid, ensuring controlled access to the transport service.

Profile Page:

The **Profile Page** in the BIET Transport App is customized based on the user's role to provide relevant features and controls. For **Students and Faculty**, it offers options to edit their profile, update their address, register for a specific bus by selecting a stop, and access modules like account settings, contact us, about us, and logout — each functioning independently to manage personal information and support. For **Admins**, the Profile Page includes all these general settings along with advanced administrative tools such as managing bus passenger lists, overseeing registered buses, and viewing detailed lists of students, faculty, and fellow admins. This role-specific design ensures efficient user management and streamlined control of the transport system.

6.3 REQUIREMENTS SPECIFICATIONS:

Minimum Software Requirements

1.	Operating systems	:	WINDOWS 11/12/XP
2.	IDE	:	1. ANDROID STUDIO SDK VERSION 17 2. JRE – version – 17 (or) Higher 3. VISUAL STUDO CODE
3.	Back end	:	NODE JS - windows-x64
4.	Front end	:	REACT NATIVE
5.	Database	:	ORACLE 0.21C

Minimum Hardware Requirements

1.	Processor	:	Intel i5
2.	RAM Capacity	:	12 GB (or) Higher
3.	SSD Capacity	:	256 GB (or) Higher

6.4 UML DIAGRAMS:

6.3.1 USECASE DIAGRAM:

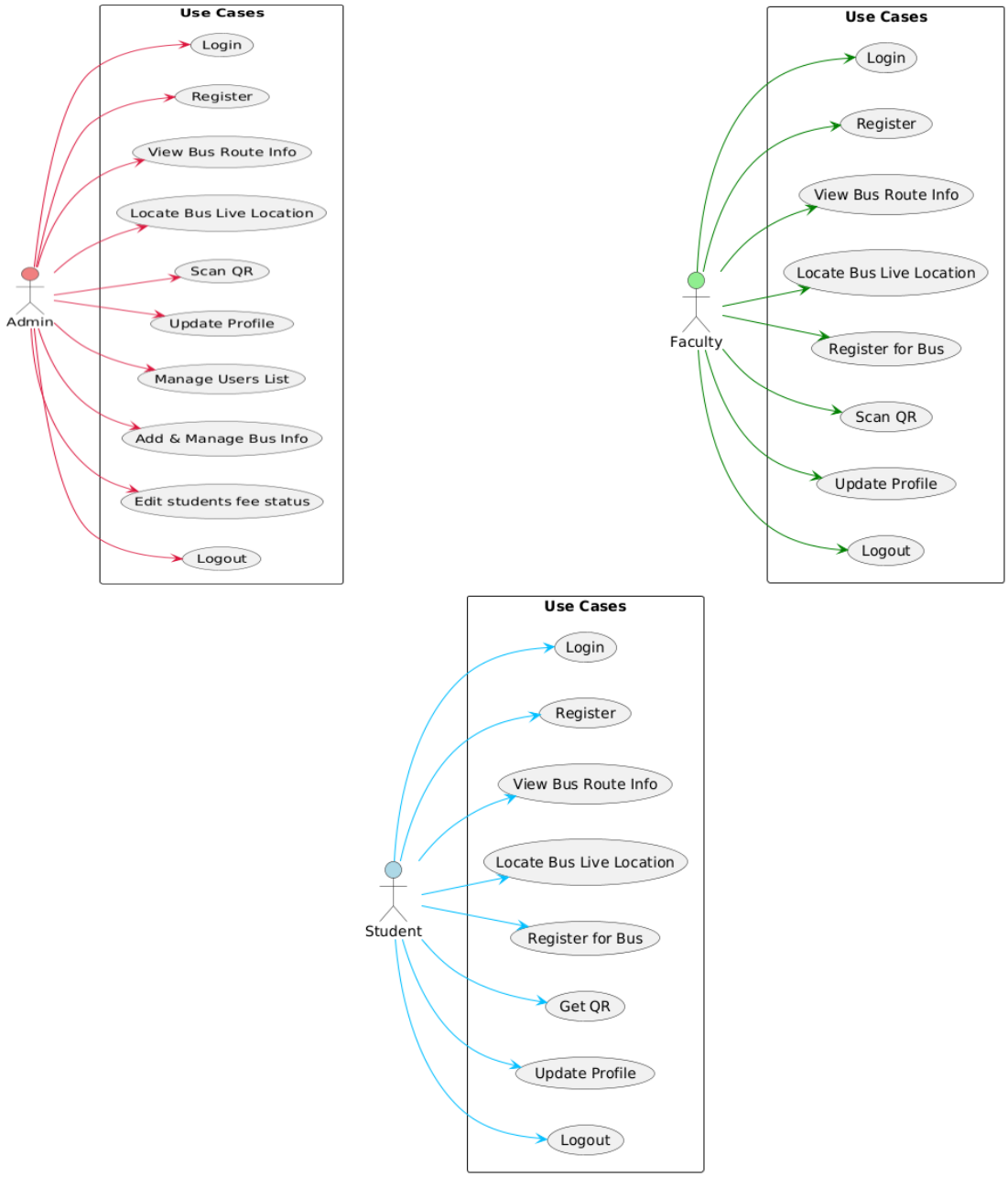


Figure 6.2: Use Case diagrams of BIET Transport Application

The use case diagram tells about the user interactions within the application, categorizing functionalities based on roles—Admin, Faculty, and Student. It encapsulates core operations such as authentication, registration, QR verification, real-time bus tracking, route management, and role-specific dashboard access.

6.3.2 CLASS DIAGRAM :

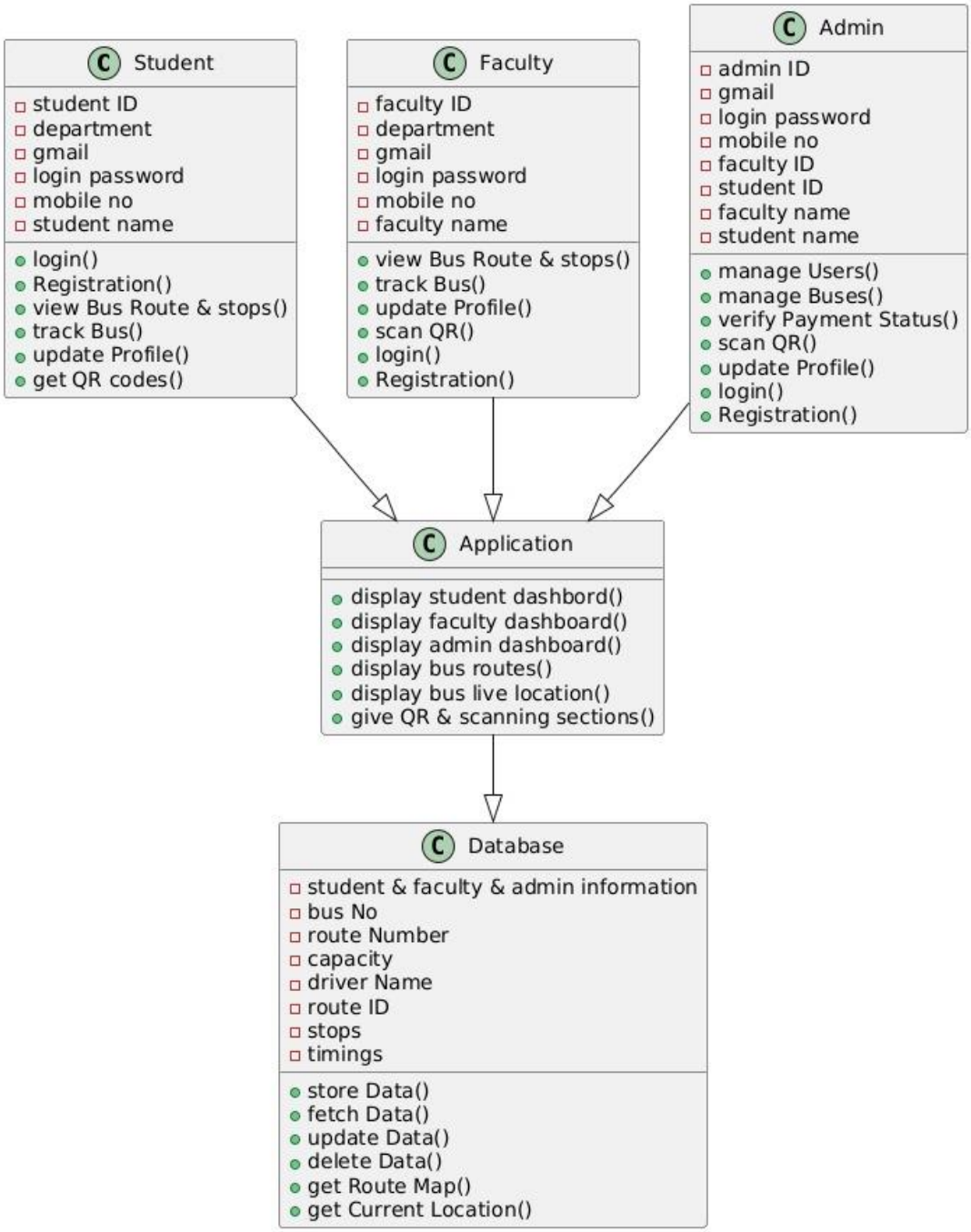


Figure 6.3: Class diagram of BIET Transport Application

This class diagram shows the set of classes, interfaces, collaboration and their relationships. This Class diagram of BIET Transport Application includes all the active classes that are used to address this static process view of a system.

6.3.3 ACTIVITY DIAGRAM*:*

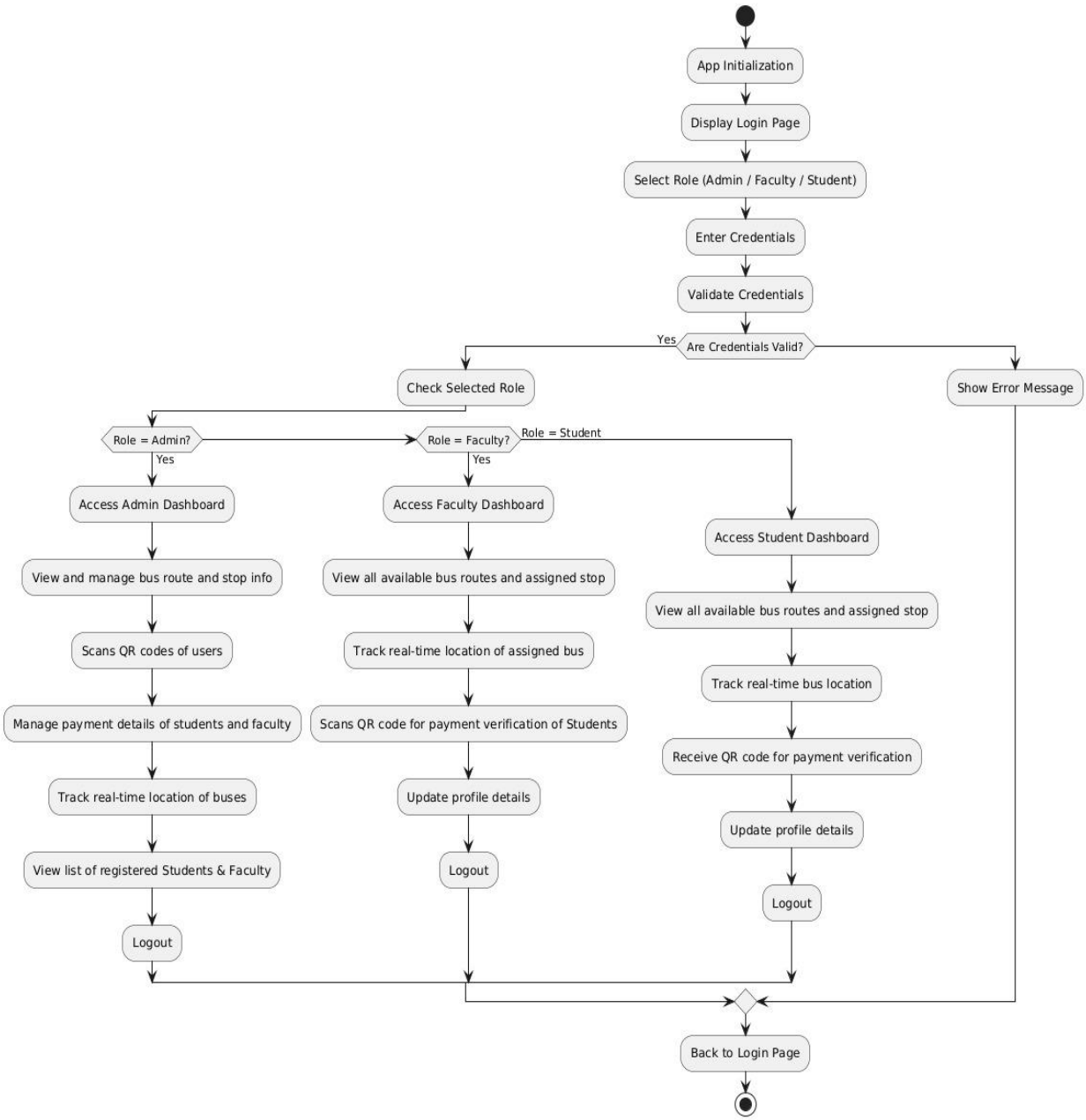


Figure 6.4: Activity diagram of BIET Transport Application.

This Activity diagram describes the flow of control of the whole system. It consists of activities and links. The flow is sequential, concurrent or branched. Activities are nothing but function of a modules. Numbers of activity diagrams are prepared to capture the entire flow in a system.

6.3.4 SEQUENCE DIAGRAM:

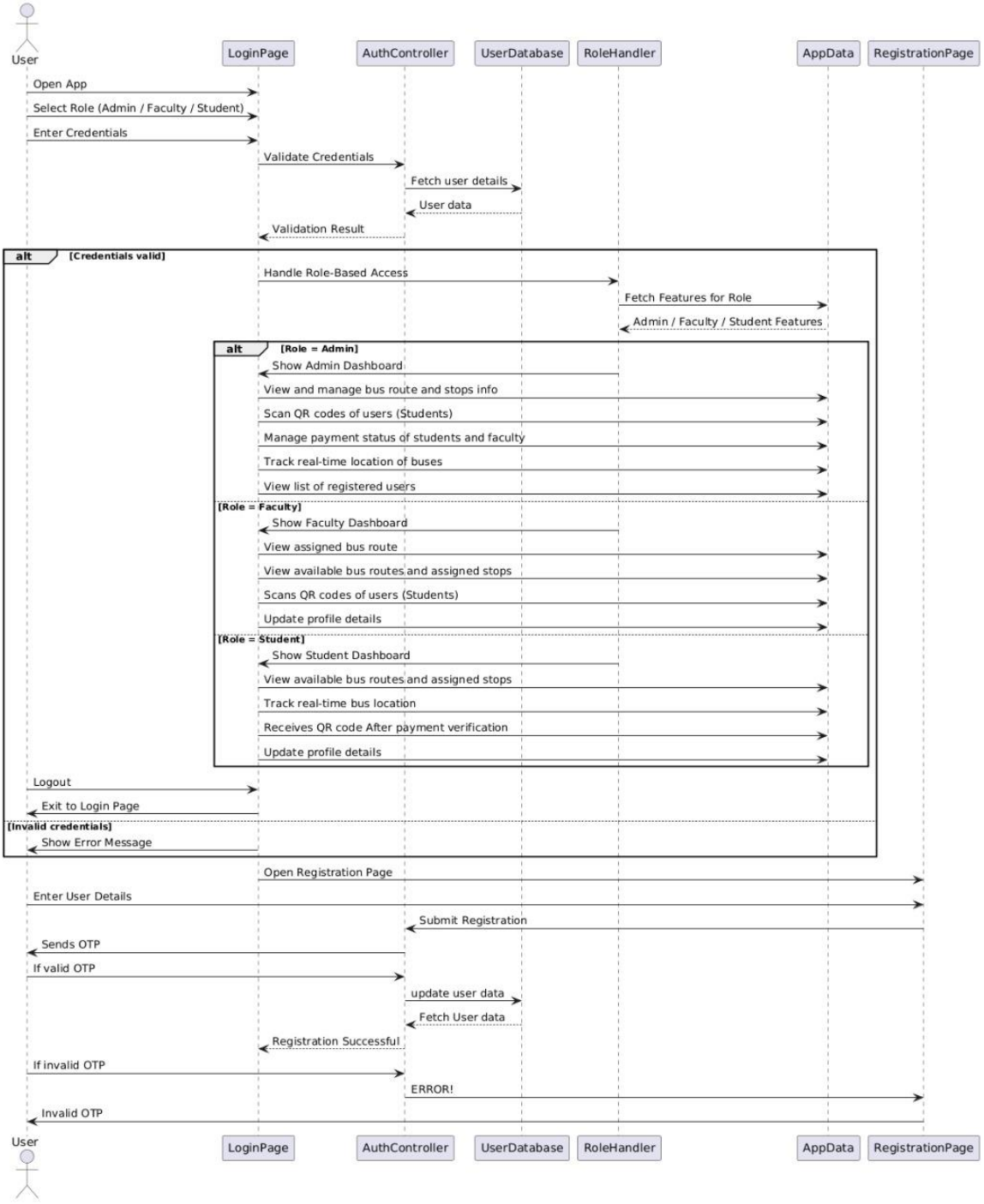


Figure 6.5: Sequence diagram of BIET Transport Application.

This sequence diagram is the interaction that emphasizes the time ordering of messages. This sequence diagram shows a set of objectives and messages set and receive by those objects. The objects are typically named or anonymous instances of other things, such as collaboration, components and nodes.

6.3.5 STATE CHART:

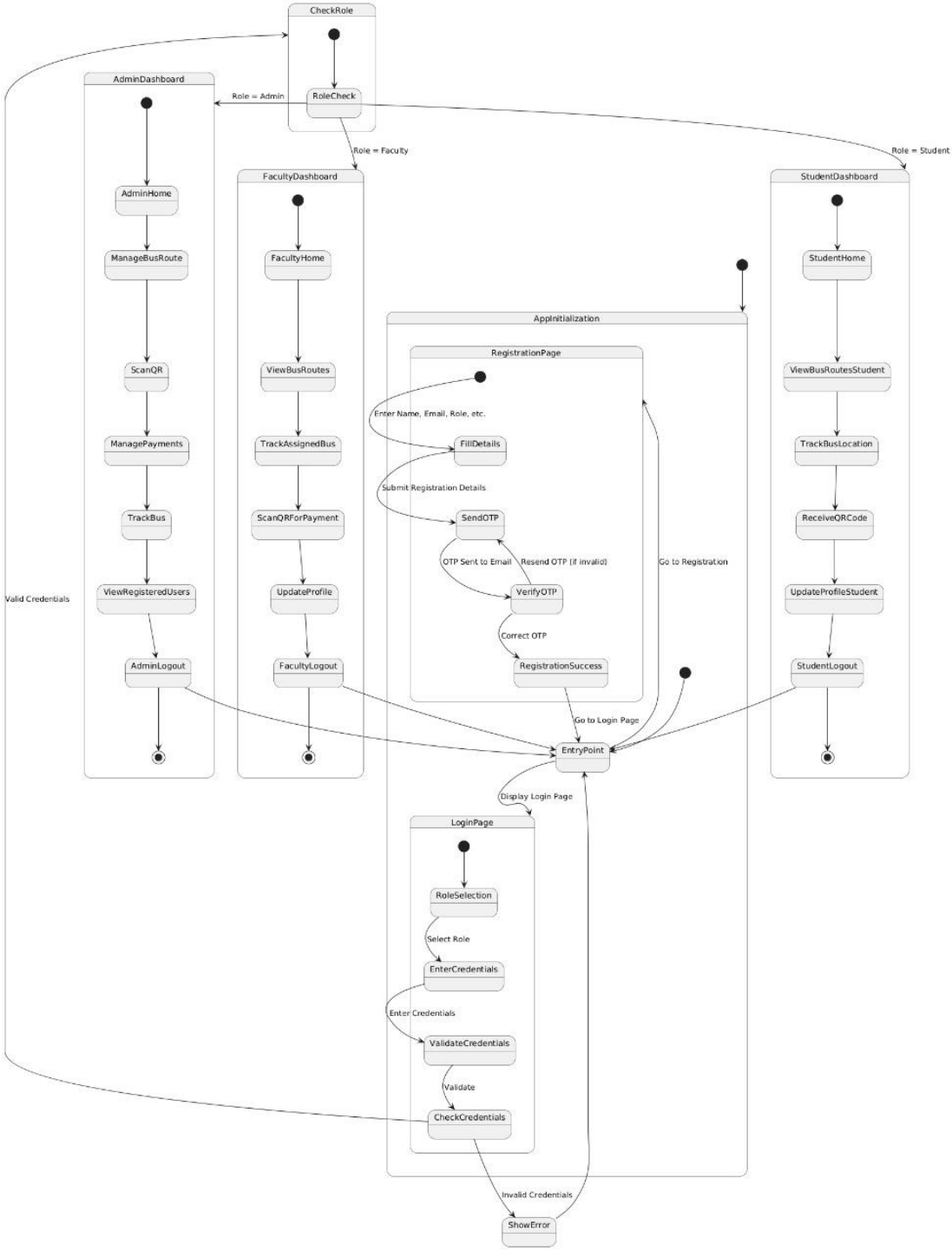


Figure 6.6: State chart of BIET Transport Application.

This state chart diagram visualizes the flow of execution from one state to another state of an object. Each state represents some unique information about the object. It represents the state of an object from the creation of an object until the object is destroyed or terminated.

7. EXPERIMENTAL STUDIES

7.1 Overview

Software testing is a process, to evaluate the functionality of a software application with an intent to find whether the developed software met the specified requirements or not and to identify the defects to ensure that the product is defect free in order to produce the quality product. As per the current trend, due to constant change and development in digitization, our lives are improving in all areas. The way we work is also changed. We access our bank online we do shop online; we order food online and many more. We rely on software's and systems. We all know that one small bug shows huge impact on business in terms of financial loss and goodwill. To deliver a quality product, we need to have Software Testing in the Software Development Process.

1. Cost effectiveness
2. Customer Satisfaction
3. Security
4. Product Quality

Dimensions of Testing

There are many different dimensions to consider:

1. Layers of the application (database, APIs, UI)
2. Scale of testing (unit, module, integration, scenario)
3. Type of testing (functional, performance, security, etc.)
4. Methodology (exploratory, scripted manual, automated)

7.2 Stages of Testing

Unit Testing

During this first round of testing, the program is submitted to assessments that focus on specific units or components of the software to determine whether each one is fully functional. The main aim of this endeavour is to determine whether the application functions as designed. In this phase, a unit can refer to a function, individual program or even a procedure, and White box testing method is usually used to get the job done. One of the biggest benefits of this testing phase is that it can be run every time a piece of code is changed, allowing issues to be resolved

as quickly as possible. It quite common for software developers to perform unit tests before delivering software to testers for formal testing.

Integration Testing

Integration testing allows individuals the opportunity to combine all of the units within a program and test them as a group. This testing level is designed to find interface defects between the modules/functions. This is particularly beneficial because it determines how efficiently the units are running together. Keep in mind that no matter how efficiently each unit is running, if they properly integrated, it will affect the functionality of the software program. In order to run these types of tests, individuals can make use of various testing methods, but the specific method that will be used to get the job done will depend greatly on the way in which the units are defined.

7.3 Test Cases

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	Login with valid credentials	Valid Id, password	Redirect to appropriate dashboard (Student/Faculty/Admin)	Should login successfully
2	Login with invalid credentials	Invalid Id/password	Error message: "Invalid credentials"	Should not login
3	Signup as a new user	Role, Id, Name, Email, DOB, Password, Mobile no	Account created, redirected to login	Should register correctly
4	Role-based redirection	Login as Admin/Faculty/Student	Dashboard specific to role	Each role sees its designated dashboard

Table 7.1: Table Of Authentication & Role Management.

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	Add/Edit bus details	Route, stop data	Changes saved to DB	Route list updates
2	Mark fee as paid	Select student ID	Fee status changed to "Paid"	QR access is triggered
3	View student/faculty lists	Open user management	List displays user data	List loads with correct data
4	View bus passenger list	Select bus	Show list of registered passengers	Correct list loads

Table 7.2: Table Of Admin Functionality.

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	View bus routes	Access Buses Page	List of routes and stops	Should display all routes
2	View bus timings	Select a route	Stop-wise arrival times shown	Should show correct schedule
3	Live location tracking	Open Map Page	Bus location marker shown on map	Location updates every 5–10 sec (pending)

Table 7.3: Table Of Bus Information & Live Tracking.

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	QR generation for paid student	Fee status = paid	Unique QR code shown	QR is generated only if paid
2	QR generation for unpaid student	Fee status = unpaid	Message: "Payment required"	QR not shown
3	QR scan at boarding	Valid QR	Show student info + access granted	Allows entry
4	Invalid QR scan	Expired/unauthorized QR	Error message shown	Denies entry

Table 7.4: Table Of Qr Code Generation & Verification.

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	View profile	Open Profile page	Show name, bus, role	Info is accurate
2	Edit profile	Update name or address	Save + refresh profile	Changes visible instantly
3	Change password	Valid old + new passwords	Success message	Password updates
4	Delete account	Confirm deletion	Account removed from DB	Account inaccessible post-delete

Table 7.5: Table Of Profile Management.

S.No	Test case	Input	Expected Output	Pass/Fail Criteria
1	Register as student	Role = Student	Redirect to student dashboard after login	Student role applied correctly
2	Register as faculty	Role = Faculty	Redirect to faculty dashboard after login	Faculty role assigned
3	Register as admin	Role = Admin	Redirect to admin panel after login	Admin rights given
4	Register with long values	300-character name input	Either trimmed or error shown	UI handles edge cases
5	Email case sensitivity check	User@Biet.edu and user@biet.edu	Treated as same	Email uniqueness enforced case-insensitively
6	Successful registration adds to DB	Register user and check DB	User record added with correct role and timestamps	Verified through DB/API

Table 7.6: Table Of Registration and Errors.

7.4 Result Analysis

1. **Test case for login**

FIELDS	INPUT	RESULT
POSITIVE CASE	Valid ID No and Password	Redirects to homepage of their role.
NEGATIVE CASE	Invalid ID No and Password	Invalid ID No or password is shown.

Table 7.7 Test case for login

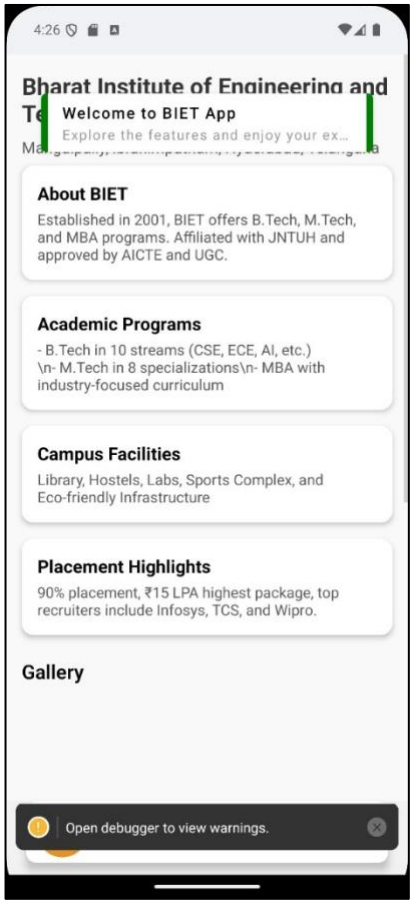


Figure 7.7.1: Positive case

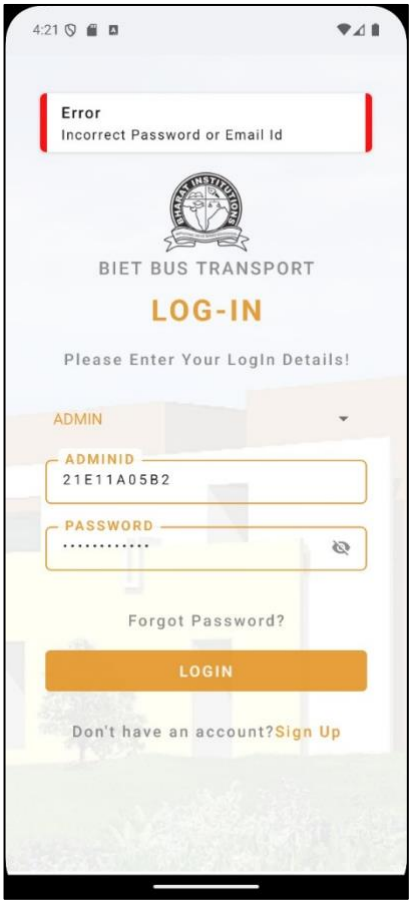


Figure 7.7.2: Negative case

2. **Test case for registration (valid OTP)**

FIELDS	INPUT	RESULT
POSITIVE CASE	Valid OTP	Redirects to login page.
NEGATIVE CASE	Invalid OTP	Invalid OTP is shown.

Table 7.8 Test case for registration

Figure 7.8.1: Positive case

Figure 7.8.2: Negative case

3. **Test case for adding bus(Admin panel)**

FIELDS	INPUT	RESULT
POSITIVE CASE	New bus details are not exist before	Bus is added successfully.
NEGATIVE CASE	New bus details are exist before	Error : Failed to add bus.

Table 7.9 Test case for adding bus

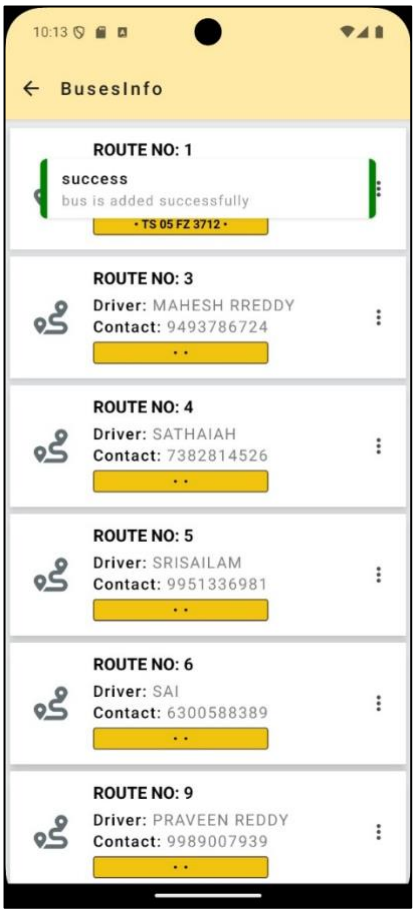


Figure 7.9.1: Positive case

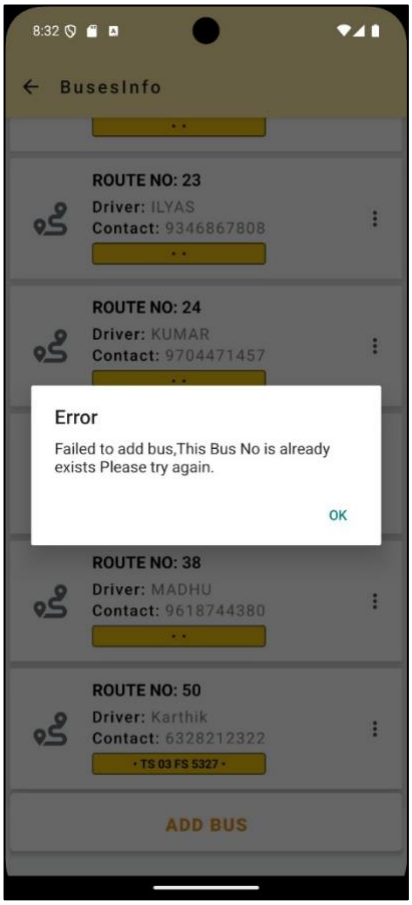


Figure 7.9.2: Negative case

4. **Test case for qr code (Student panel)**

FIELDS	INPUT	RESULT
POSITIVE CASE	If fees is paid and get verified by the admin then the admin change the student status is to paid	QR code is visible.
NEGATIVE CASE	If fees is not paid or not get verified by the admin then the status will remains same	Invalid QR is shown.

Table 7.10 Test case for qr code

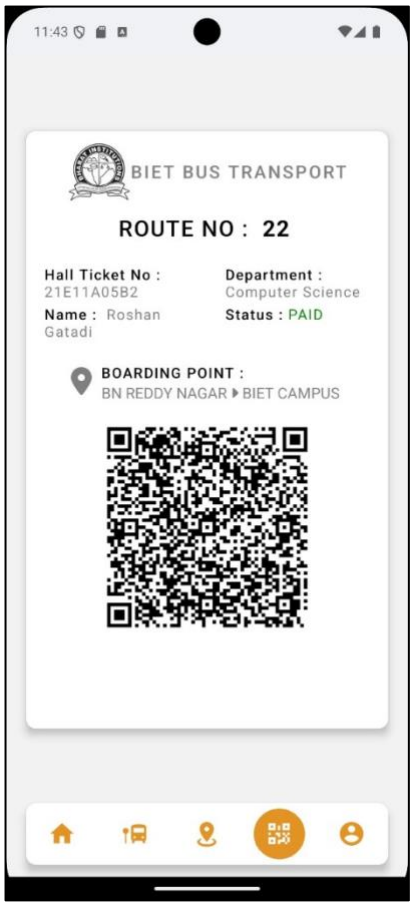


Figure 7.10.1: Positive case

Figure 7.10.2: Negative case

5. **Test case for qr code verification (Admin / Faculty panel)**

FIELDS	INPUT	RESULT
POSITIVE CASE	When admin / faculty scans the qr code of a valid student.	Student details is visible along with photo and student details.
NEGATIVE CASE	When admin / faculty scans the qr code of a invalid student.	Invalid QR is shown.

Table 7.11 Test case for qr code verification

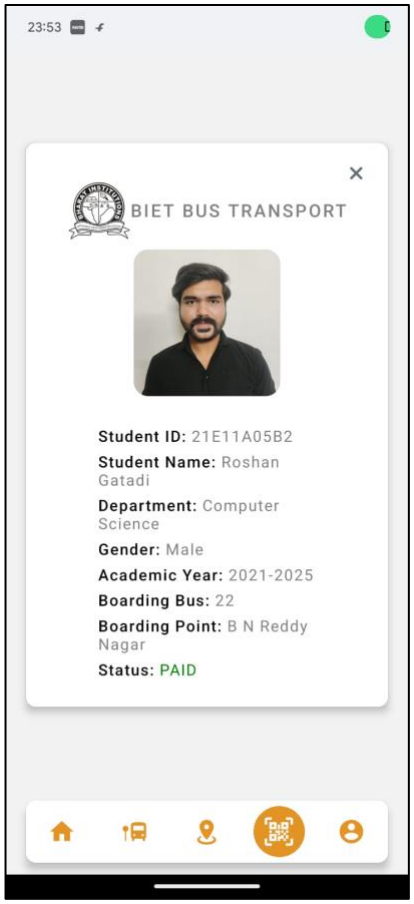


Figure 7.11.1: Positive case case

Figure 7.11.2: Negative

6. **Test case for adding of users into list (Admin)**

FIELDS	INPUT	RESULT
POSITIVE CASE	When admin add single or multiple new users in the list (Student / Faculty). (or)	New user / users are added is shown along with their roll no.
NEGATIVE CASE	When admin add single or multiple old users in the list (Student / Faculty).	User / users already exist is shown along with their roll no.

Table 7.12 Test case for adding of users into list.

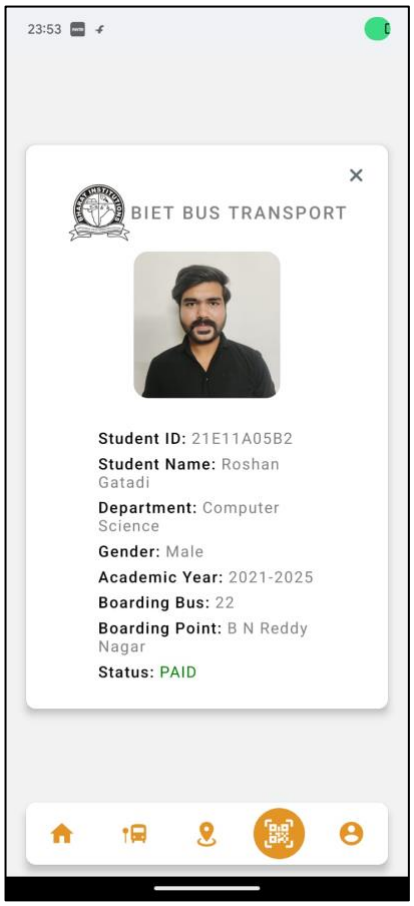


Figure 7.12.1: Positive case

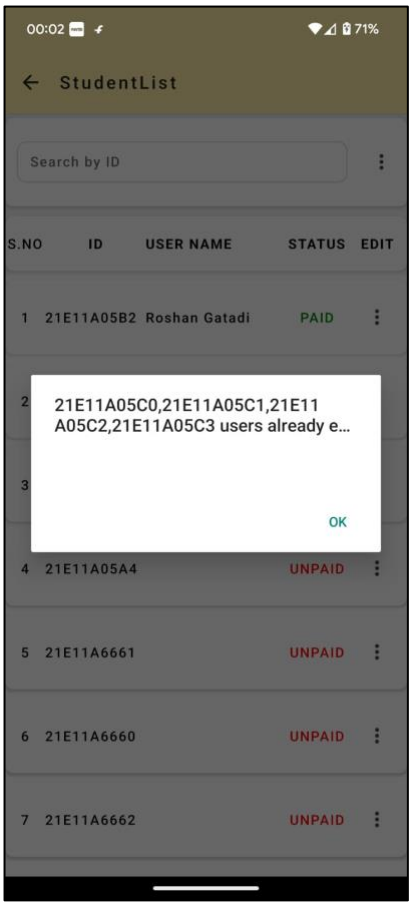


Figure 7.12.2: Negative case

The BIET Transport Application was developed to address inefficiencies in the current college transport system by providing a digital, real-time, and secure platform for students, faculty, and administrators. The application was subjected to comprehensive testing across different layers of functionality, and the results are summarized below:

Functional Accuracy

The testing phase of the **BIET Transport App** covered all major modules to ensure the system functions as intended and delivers a smooth user experience. Core features such as **Login**, **User Registration**, **Bus Tracking Interface**, **QR Code Generation and Scanning**, and **Profile Management** were carefully tested against predefined functional requirements and user scenarios. Each of these components performed reliably, contributing to the overall usability and security of the application. The **Login Module** successfully authenticated users based on their credentials and roles—*Student*, *Faculty*, or *Admin*—and redirected them to the appropriate dashboard with access tailored to their permissions. This validated the role-based access control, ensuring data privacy and task separation across user types.

The **QR Code functionality** also worked effectively, generating unique codes for fee-paid students and enabling real-time verification through QR scanning by faculty and admin users. The **Profile Management** module allowed users to update personal information, manage settings, and register for bus routes based on location, all of which functioned as expected during testing. The **Registration Page** successfully captured new user data and assigned correct role-based access after submission.

However, one key module—**real-time GPS live tracking of buses**—is still under development and was not included in the final round of testing. This feature is critical to improving the user experience by allowing students and faculty to track the current location of their bus and plan their departures more efficiently. The development team is actively working to complete and integrate this feature using GPS and map APIs, ensuring that once implemented, it will provide accurate and responsive live location updates. Once completed, this module will undergo full testing to ensure it aligns with the reliability and performance standards of the rest of the application.

Performance and Reliability

The **BIET Transport App** demonstrated strong performance and stability across a range of Android devices, operating smoothly without any significant crashes, lag, or compatibility

issues. One of the key features, **QR code-based boarding verification**, functioned precisely as intended—ensuring that only students who had paid their transportation fees could generate and present valid QR codes for bus access. This mechanism effectively prevented unauthorized usage and streamlined the boarding process.

Furthermore, the app's backend architecture supported **real-time data updates** using **Firebase**, ensuring that information such as user status, route changes, and QR validations were instantly synchronized without data loss or inconsistency. Even in scenarios where the device temporarily lost internet connectivity, the integration of a **local database** ensured uninterrupted access to critical data, maintaining app functionality and allowing users to continue using essential features offline until connectivity was restored. This combination of real-time responsiveness and offline resilience contributed to a dependable and user-friendly experience.

User Experience

The app's user interface was designed to be intuitive and user-friendly, with clear segmentation based on user roles. Students were able to easily navigate through the app to check bus timings and route details relevant to their commute. Faculty members had streamlined access to monitor the routes and buses that concerned them, allowing for efficient oversight. Meanwhile, admins were provided with comprehensive controls and management tools within the interface, enabling them to effectively oversee the entire transport system. This role-based UI design ensured that each user group could quickly find and use the features most important to them, enhancing overall usability and satisfaction.

Security and Access Control

Role-Based Access Control (RBAC) was implemented to ensure that sensitive operations—such as user management, route modifications, and fee verification—were accessible exclusively to administrators, thereby maintaining strict security and operational integrity. Additionally, linking QR codes with students' payment status added an important layer of security, significantly minimizing the risk of unauthorized access to transport services and ensuring that only eligible, fee-paid students could board the buses.

Sample Code:

```
import { ActivityIndicator, Dimensions, StatusBar, StyleSheet, Text,
View, Alert } from 'react-native'
import React, { useEffect, useState } from 'react'
import { NavigationContainer, NavigationIndependentTree, useNavigation }
from '@react-navigation/native'
import { createStackNavigator, StackScreenProps } from '@react-
navigation/stack'
import LogIn from './components/LogIn';
import Signup from './components/Signup';

import Register from './components/Register';
import Faculty from './components/Faculty';
import Admin from './components/Admin';
import AsyncStorage from '@react-native-async-storage/async-storage'
import { BottomTabBarProps, createBottomTabNavigator } from '@react-
navigation/bottom-tabs';
import Icon from 'react-native-vector-icons/MaterialCommunityIcons';
import Profile from './components/Profile';
import Location from './components/Location';
import QR from './components/QR';
import Timings from './components/Timings';
import HomeScreen from './components/HomeScreen';
import ContactUs from './components/ProfileComponent/ContactUs';
import Password from
'./components/ProfileComponent/AccountComponents/Password';
import Delete from
'./components/ProfileComponent/AccountComponents/Delete';
import AccountSettingsScreen from
'./components/ProfileComponent/AccountSettings';
import Address from './components/ProfileComponent/Address';
import BusDetails from './components/ProfileComponent/BusDetails';
import BusRegister from './components/ProfileComponent/BusRegister';
import AboutUs from './components/ProfileComponent/AboutUs';
import LogOut from './components/ProfileComponent/LogOut';
import EditProfile from './components/ProfileComponent/EditProfile';
import { ipAddress } from './components/ipAddress';
import axios from 'axios';
import Scanner from './components/Scanner';
import BusesInfo from './components/ProfileComponent/BusesInfo';
import BusStops from
'./components/ProfileComponent/BusInfoComponents/BusStops';
import UserList from './components/ProfileComponent/UserList';
import AdminList from './components/ProfileComponent/AdminList';
import StudentList from './components/ProfileComponent/StudentList';
import FacultyList from './components/ProfileComponent/FacultyList';
import OTP from './components/OTP';
```

```
import Toast, { BaseToast, ErrorToast } from 'react-native-toast-
message';
import { get } from 'react-
native/Libraries/TurboModule/TurboModuleRegistry';
import ForgotPassword from './components/ForgotPassword';

const toastConfig = {
  success: (props) => {
    <BaseToast
      {...props}
      style={{
        borderLeftColor: 'green',
        borderRightColor: 'green',
        borderLeftWidth: 7,
        borderRightWidth: 7,
        width: '90%',
        height: 70
      }}
      contentContainerStyle={{ paddingHorizontal: 15 }}
      text1Style={{ fontSize: 17, fontWeight: '700', letterSpacing: 1.2
}}
    />
  },
  error: (props) => {
    <ErrorToast
      {...props}
      text1Style={{ fontSize: 17, fontWeight: '400', letterSpacing: 1.2
}}
      text2Style={{ fontSize: 17, fontWeight: '400', letterSpacing: 1.2
}}
      style={{
        borderLeftColor: 'red',
        borderRightColor: 'red',
        borderLeftWidth: 7,
        borderRightWidth: 7,
        width: '90%',
        height: 70
      }}
      contentContainerStyle={{ paddingHorizontal: 10 }}
    />
  }

}

const { width } = Dimensions.get('window');
const iconSize = width * 0.07;
```

```
const Stack = createStackNavigator()

const Tab = createBottomTabNavigator();
function TabBar() {
  const [data, setData] = useState(null);
  async function getData() {
    const data = await AsyncStorage.getItem('isLoggedIn');
    if (data === "true") {
      setIsLoggedIn(true);
    }
    else {
      setIsLoggedIn(false);
    }

  }

  async function getUser() {
    const token = await AsyncStorage.getItem("token");
    // console.log(token,"tabbaaarrrrr")
    axios.get(http://${ipAddress}:8005/getuser, {
      headers: {
      Authorization: Bearer ${ token }
      }
})
      .then(res => {
  // console.log(res.data,"res")
  setData(res.data)
  // console.log(user)
  // console.log('hjey')
})
  .catch(err => {
    console.log(err)
    // Alert.alert('Error', 'Something went wrong')
  })
  }


useEffect(() => {
  const fetchData = async () => {
    await getData();
    await getUser();
    setLoading(false);
  };
  fetchData();
}, [])
if (data === null) {
  getUser()
```

```
  return <Text>Loading</Text>; // Show a loading state
}

return (
  <Tab.Navigator
    screenOptions={({ route }) => ({
      tabBarIcon:
        ({ focused, color, size }) => {
          let iconName;
          switch (route.name) {
            case 'HomeScreen':
              iconName = 'home'
              break;
            case 'Timings':
              iconName = 'bus-stop'
              break;
            case 'Location':
              iconName = 'map-marker-radius'
              break;
            case 'QR':
              iconName = 'qrcode'
              break;
            case 'Scanner':
              iconName = 'qrcode-scan'
              break;
            case 'Profile':
              iconName = 'account-circle'
              break;

            default: iconName = 'default'
              break;
          }
          return <Icon name={iconName} size={iconSize} color={focused ?
'white' : '#E39424'} />

        },
      tabBarActiveBackgroundColor: '#E39424',
      tabBarItemStyle: {
        // marginBottom:'3%',
        marginHorizontal: '2.9%',
        marginVertical: '1.5%',
        borderRadius: '50%',
        zIndex: 1
      },
      tabBarIconStyle: {
        borderRadius: '50%',
        flex: 1
      },
```

```
      tabBarStyle: {
        // flex:1,
        justifyContent: 'center',
        alignItems: 'center',
        zIndex: 1,
        height: '7.5%',
        width: '90%',
        // backgroundColor: 'transparent',
        marginBottom: '3%',
        marginHorizontal: 'auto',
        borderTopWidth: 0,
        borderRadius: 10,
        shadowOpacity: 0.5,
        shadowRadius: 10,
        shadowOffset: { width: 0, height: 10 },
        elevation: 5,
      },
      headerShown: false,
      tabBarShowLabel: false,
      backgroundColor: 'transparent',
      // headerTransparent:true,

    }
    )}
  >
    <Tab.Screen name="HomeScreen" component={HomeScreen}
    />
    <Tab.Screen name="Timings" component={Timings} />
    <Tab.Screen name="Location" component={Location} />
    <Tab.Screen name={data.HALLTICKETNO ? "QR" : "Scanner"}
component={data.HALLTICKETNO ? QR : Scanner} initialParams={{ data }} />
    <Tab.Screen name="Profile" component={Profile} initialParams={{ data
}} />

  </Tab.Navigator>
);
}

export default function App() {
  const [isLoggedin, setIsLoggedIn] = useState(false)
  const [data, setData] = useState(null);

  // const [userData, setUserData] = React.useState()
  const [loading, setLoading] = useState(true)
  async function getData() {
    const token = await AsyncStorage.getItem("token");
    const data = await AsyncStorage.getItem('isLoggedIn');
    // console.log(data)
```

```
    // console.log(token)
    if (data === "true") {
      setIsLoggedIn(true);
    }
    else {
      setIsLoggedIn(false);
    }

  }

  async function getUser() {
    const token = await AsyncStorage.getItem("token");
    // console.log(token)
    axios.get(http://${ipAddress}:8005/getuser, {
      headers: {
      Authorization: Bearer ${ token }
      }
})
      .then(res => {
  // console.log(res.data)
  setData(res.data)
})
  .catch(err => {
    console.log(err, "In App")
    if (err.response.status === 403) {
      // Alert.alert('Error', 'Session expired, please login again')
      AsyncStorage.setItem('isLoggedIn', 'false')
      AsyncStorage.setItem('token', '')
      setIsLoggedIn(false)
      // getUser()
    }
  })
  }


useEffect(() => {
  const fetchData = async () => {
    await getData();
    await getUser();
    setLoading(false);
  };
  fetchData();
}, [])


return (
  <NavigationIndependentTree>
```

```
    <StatusBar backgroundColor={'#f1f2f6'} barStyle={'dark-
content'}></StatusBar>
    <NavigationContainer>
      {loading ? (<View style={{ flex: 1, justifyContent: 'center',
alignItems: 'center' }}><ActivityIndicator size="large" color="#E39424"
/></View>) :
        <Stack.Navigator initialRouteName={isLoggedin ? 'TabBar' :
'LogIn'} screenOptions={{ headerShown: false }}>
          <Stack.Screen name='LogIn' component={LogIn}></Stack.Screen>
          <Stack.Screen name='OTP' component={OTP}></Stack.Screen>
          <Stack.Screen name='Register'
component={Register}></Stack.Screen>
          <Stack.Screen name='Admin' component={Admin}></Stack.Screen>
          <Stack.Screen name='Faculty'
component={Faculty}></Stack.Screen>
          <Stack.Screen name='Signup' component={Signup}></Stack.Screen>
          <Stack.Screen name='ForgotPassword'
component={ForgotPassword}></Stack.Screen>
          <Stack.Screen name='TabBar' component={TabBar}></Stack.Screen>
          <Stack.Screen name="Profile" component={Profile}
initialParams={{ data }} />
          <Stack.Screen name="EditProfile" component={EditProfile}
options={{
            headerStyle: { backgroundColor: '#ffeaa7', elevation: 0, },
headerShown: true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="Address" component={Address} options={{
            headerTitle: 'Saved Address', headerShown: true, headerStyle:
{ backgroundColor: '#ffeaa7' }, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="BusRegister" component={BusRegister}
options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
```

```
          <Stack.Screen name="BusDetails" component={BusDetails}
options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="BusesInfo" component={BusesInfo} options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="BusStops" component={BusStops} options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="UserList" component={UserList} options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="StudentList" component={StudentList}
options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="FacultyList" component={FacultyList}
options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
```

```
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="AdminList" component={AdminList} options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />

          {/* <Stack.Screen name="AccountSettings"
component={AccountSettingsScreen}
options={{headerStyle:{backgroundColor:'#f1f2f6'}}} /> */}
          <Stack.Screen name="ContactUs" component={ContactUs} options={{
            headerStyle: { backgroundColor: '#ffeaa7', elevation: 0, },
headerShown: true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="AboutUs" component={AboutUs} options={{
            headerStyle: { backgroundColor: '#ffeaa7' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />
          <Stack.Screen name="LogOut" component={LogOut} options={{
            headerStyle: { backgroundColor: '#f1f2f6' }, headerShown:
true, headerTitleStyle: {
              fontSize: 18,
              fontWeight: '400',
              letterSpacing: 2.5
            }
          }} />

          <Stack.Screen
            name="AccountSettings"
            component={AccountSettingsScreen}
            options={{
              headerShown: true, headerStyle: { backgroundColor:
'#ffeaa7' }, headerTitleStyle: {
                fontSize: 18,
                fontWeight: '400',
```

```
                letterSpacing: 2.5
              }
            }} />
          <Stack.Screen
            name="ChangePassword"
            component={Password}
            initialParams={{ data }}
            options={{
              title: 'Change Password', headerShown: true, headerStyle: {
backgroundColor: '#ffeaa7' }, headerTitleStyle: {
                fontSize: 18,
                fontWeight: '400',
                letterSpacing: 2.5
              }
            }} />
          <Stack.Screen
            name="DeleteAccount"
            component={Delete}
            initialParams={{ data }}
            options={{
              title: 'Delete Account', headerShown: true, headerStyle: {
backgroundColor: '#ffeaa7' }, headerTitleStyle: {
                fontSize: 18,
                fontWeight: '400',
                letterSpacing: 2.5
              }
            }} />

          {/* <Stack.Screen name='HomeScreen'
component={HomeScreen}></Stack.Screen> */}
        </Stack.Navigator>}
      <Toast config={toastConfig} />
    </NavigationContainer>
  </NavigationIndependentTree>
)
}

const styles = StyleSheet.create({})
```

RESULTS:

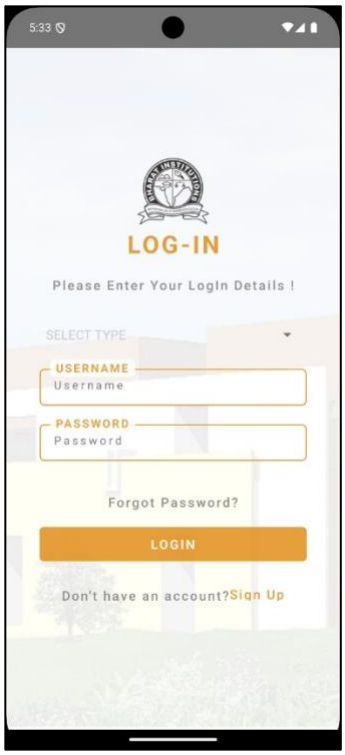


Figure 7.1: Login page.

Login Page:
user can login in to the app by selecting their role and by entering their id no and password. If they doesn't have an account then they can get registered into the app by pressing sign-up button.

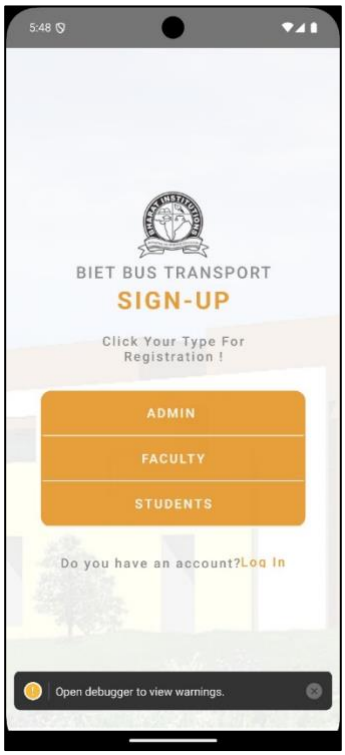
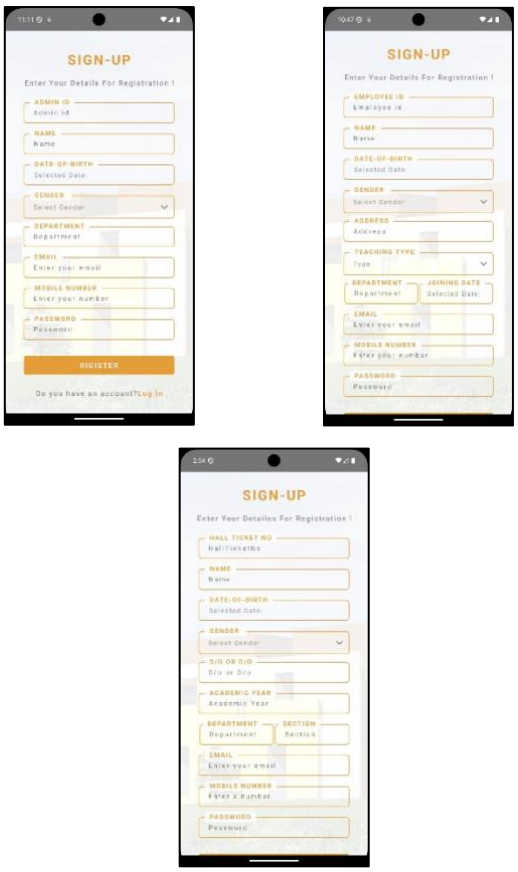


Figure 7.2: Sign-up page.

Sign-Up page:
This is the page where user can select their role and get sign-up. Based upon the selected role the sign-up page is loaded. By providing the valid credentials the users can get sign-in to the app.



Registration Pages:
Sign-up pages of admin, faculty and students according to the roles they have selected. based upon their selected role different sign-up pages are loaded.

Figure 7.3: Registration pages of different Roles.

Home Page:
Home page contains the certificates, rewards and achievements of the colleges and a link which will direct the users to the email and phone of the college and also user can pay the fees in the sbi collect which is directly linked to the home section.

Figure 7.4: Home page.

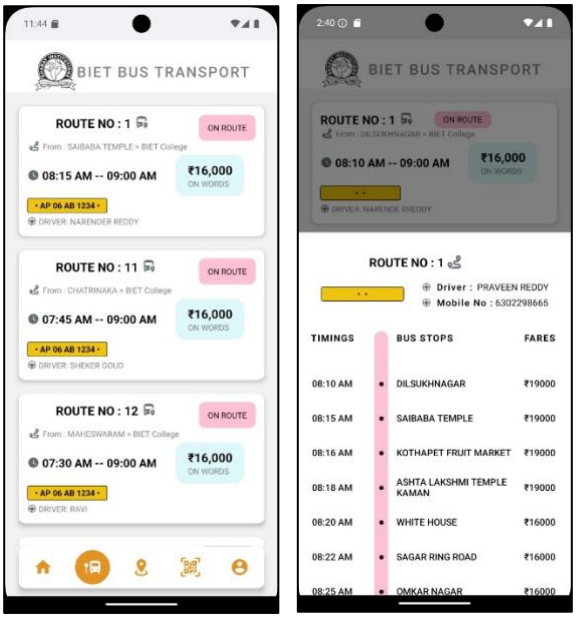


Figure 7.5: Buses-info page.

Buses-info Page:
This are the buses-info page of all the buses which includes the information of stops, fares and time of arrival of each particular bus.

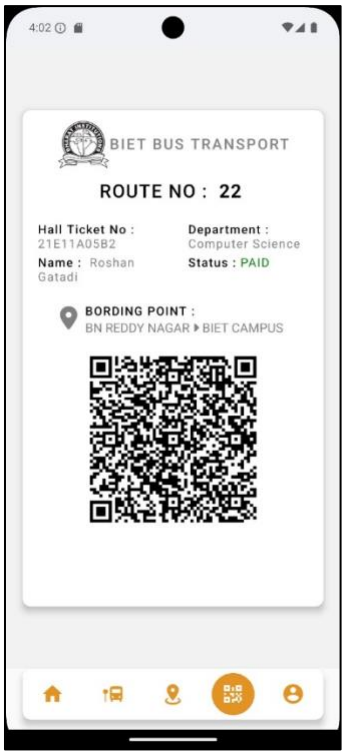


Figure 7.6: Qr , Scanner page.

QR, Scanner Page:
This is a qr code section for student and scanner section for admin and faculty .the QR is loaded for the fee paid students and admin and facuty can scan their Qr to verify whether the student is a valid or not.

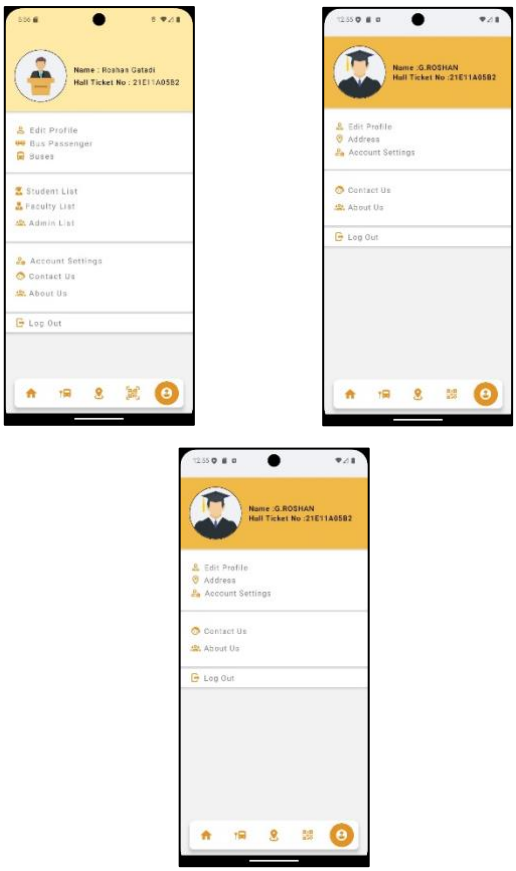


Figure 7.7: Profile page of different roles.

Profile Pages:
Profile page of admin, faculty, and student. This is the section where student can edit their details and manage their accounts and if there is any problem they can contact to the transport service department.

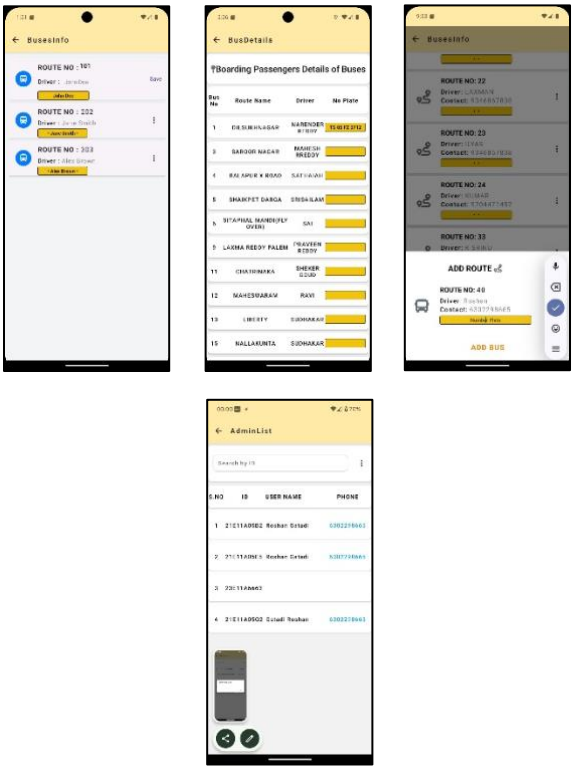


Figure 7.8: admin management page.

Admin Management Page:
Admin's bus management and passengers Lists pages. In this pages admin can manage bus details, add new buses, view the list of boarding students of each bus etc and can also manage the fee status of a student.

8. CONCLUSION AND FUTURE WORK

The **BIET Transport Application** was developed to streamline and modernize the transportation system within the Bharat Institute of Engineering and Technology. By integrating real-time GPS tracking, cloud-based data handling, and QR code-based verification, the app successfully addresses major challenges faced by traditional transport systems—such as lack of transparency, manual verification, and delayed communication.

The role-based access system ensures that each user—student, faculty, or administrator—has access only to the features relevant to their responsibilities. Students can view live bus locations, faculty can monitor specific routes, and admins have full control over buses, users, and route configurations. The app's integration with a cloud database enables real-time data synchronization, providing a consistent and reliable user experience across all devices.

Security is enhanced by allowing only fee-verified students to generate QR codes for bus access, reducing unauthorized usage. The intuitive user interface, combined with efficient background services like location tracking and connectivity monitoring, makes the app both powerful and user-friendly.

In future iterations, several enhancements can be implemented to further improve the system. Features like in-app notifications or alerts for route changes, AI-based bus arrival time prediction, and integration with biometric systems can add more intelligence and convenience. A web-based admin dashboard could be developed for better administrative control and reporting. Offline QR code caching and emergency alert features could improve safety and usability in low-connectivity areas.

Additionally, analytics tools can be integrated to generate usage reports, optimize route planning, and track peak load times. Expanding the system to support inter-campus transport or ride-sharing for students could further increase its impact.

Overall, the project lays a strong foundation for a scalable, secure, and intelligent transport system, contributing to a more efficient and digitally empowered campus environment.

REFERENCES

[1.] **M. H. Rahman, A. Hossain, F. Kabir, and S. Iqbal**, "**An IoT-Based Intelligent Transport System for University Campuses,**" *IEEE Access*, vol. 11, pp. 6843–6855, Jan. 2023.

[2.] **P. Jain and R. Singh**, "**Analysis of Real-Time Transport Solutions Using MQTT and Firebase**," *2023 International Conference on Computing and Communication Systems (I3CS)*, Kolkata, India, 2023.

[3.] **M. S. Rehman, S. Hafeez and A. Ghafoor**, "**Design and Implementation of a QR Code Based Smart Attendance System**," *2022 9th International Conference on Future Internet of Things and Cloud (FiCloud)*, Rome, Italy, 2022, pp. 84–90.

[4.] **T. Sharma and R. Jain**, "**Smart Bus Tracking System Using Cloud Technology**," *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 10, no. 5, pp. 95–98, June 2021.

[5.] **J. K. Kodali and K. Kiran**, "**Cloud-based Bus Tracking and Passenger Alert System**," *International Journal of Advanced Research in Computer and Communication Engineering*, vol. 10, no. 7, pp. 199–203, July 2021.

[6.] **S. Gupta and V. Singh**, "**IoT-based real-time bus tracking system using GPS and Firebase**," *2021 International Conference on Intelligent Engineering and Management (ICIEM)*, London, UK, 2021, pp. 15–20.

[7.] **P. Chawla and M. Kumari**, "**Role-Based Access Control in Mobile Applications for Academic Institutions**," *2021 International Conference on Computing, Communication, and Intelligent Systems (ICCCIS)*, Greater Noida, India, 2021, pp. 462–467.

[8.] **A. Khan and P. Kumar**, "**A GPS-Based Bus Tracking System Using Firebase Realtime Database**," *International Journal of Engineering Research & Technology (IJERT)*, vol. 10, no. 6, pp. 235–239, June 2021.

[9.] **A. R. Dar and S. A. Lone**, "**Design and development of campus shuttle tracking app using Google Maps API and Firebase**," *International Journal of Computer Applications*, vol. 183, no. 8, pp. 10–14, 2021.

[10.] **V. Patel and J. Joshi**, "**Design of an Android Application for Smart Bus Management System**," *International Research Journal of Engineering and Technology (IRJET)*, vol. 8, no. 4, pp. 1466–1470, April 2021.